

TP02 — Jeux en situation d'adversité

Documentation du projet — Chess AI (MinMax & élagage Alpha-Bêta)

Module : Jeux en situation d'adversité

Enseignant : Adrien Escourrou

Date limite de rendu : 17/07/2026

1. Contexte et besoin

Ce projet a été réalisé dans le cadre du TP02 noté du module « Jeux en situation d'adversité ». L'objectif pédagogique est de comprendre et d'implémenter l'algorithme MinMax, associé à l'élagage Alpha-Bêta, pour permettre à une intelligence artificielle de choisir un coup optimal dans un jeu à deux joueurs à information parfaite.

Le jeu choisi est les échecs : un jeu à somme nulle, à information complète, avec un espace de recherche qui croît très rapidement avec la profondeur (facteur de branchement moyen d'environ 35 coups légaux par position). Il constitue donc un bon terrain d'application pour observer concrètement l'intérêt de l'élagage Alpha-Bêta par rapport à un MinMax naïf.

Le besoin fonctionnel est de proposer une IA autonome, capable de jouer une partie complète contre un joueur humain via une interface graphique, en s'appuyant uniquement sur son propre calcul (pas de moteur d'échecs externe type Stockfish).

2. Fonctionnalités

2.1 Cœur du jeu

- Échiquier interactif en Pygame : sélection d'une pièce à la souris, affichage des coups légaux associés, déplacement par clic sur la case cible.
- Le joueur humain joue toujours les Blancs ; l'IA joue automatiquement les Noirs dès que c'est son tour.
- Détection de fin de partie (échec et mat, pat, nulle par matériel insuffisant, règle des 50 coups, répétition de position) et affichage du résultat.
- Promotion automatique en Dame.

2.2 Intelligence artificielle (ai/custom_ai.py)

- Algorithme MinMax récursif avec élagage Alpha-Bêta (coupure dès que $\alpha \geq \beta$).
- Profondeur de recherche dynamique : elle augmente automatiquement lorsque le nombre de pièces restantes diminue (l'espace de recherche étant plus restreint en finale, l'IA peut se permettre de regarder plus loin).

- Fonction d'évaluation composite (voir §3.2) combinant matériel et critères positionnels.

2.3 Fonctionnalités annexes

- Historique des coups joués (notation SAN) affiché dans le panneau latéral.
- Graphique du temps de réflexion de l'IA coup par coup (Matplotlib), accessible via le bouton « Statistiques ».
- Visualiseur d'arbre de recherche (bonus) : export HTML autonome et interactif de l'arbre MinMax réellement exploré pour le dernier coup (nœuds, valeurs alpha/bêta, coup associé, branches élaguées), navigable au clic/molette sans connexion internet.
- Tests unitaires automatisés (pytest) vérifiant la légalité des coups renvoyés par l'IA.

3. Choix techniques

3.1 Bibliothèques et architecture

python-chess	Représentation de l'échiquier, génération des coups légaux, notation SAN, détection de fin de partie
pygame	Interface graphique (échiquier, panneau latéral, gestion des clics)
matplotlib	Graphique du temps de réflexion de l'IA
pytest	Tests unitaires de l'IA

L'architecture sépare clairement la logique de jeu et d'affichage (game.py, renderer.py, menu.py, stats.py) de la logique de décision de l'IA, isolée dans ai/custom_ai.py. Cette séparation garantit que l'algorithme MinMax peut être testé et modifié indépendamment du reste de l'application. Le module ai/tree_logger.py, purement observationnel, enregistre les nœuds visités pendant la recherche sans influencer la décision de l'IA, pour alimenter le visualiseur d'arbre (tree_html_export.py).

```

Echec_MinMax/
├── main.py          # point d'entrée
├── menu.py          # menu de sélection
├── game.py          # boucle de jeu, tour par tour, appel de l'IA
├── renderer.py      # affichage Pygame (échiquier + panneau latéral)
├── stats.py         # graphique matplotlib (temps de calcul IA)
├── tree_html_export.py # export de l'arbre de recherche en HTML
├── tree_visualizer.py # utilitaires de visualisation de l'arbre
├── config.py        # constantes (couleurs, tailles, profondeur par défaut)
├── ai/
│   ├── custom_ai.py # <-- MinMax + élagage Alpha-Bêta + évaluation
│   └── tree_logger.py # enregistrement de l'arbre exploré (pour la visualisation)
├── tests/
│   └── test_custom_ai.py # tests unitaires
└── requirements.txt

```

3.2 Fonction d'évaluation

La position est évaluée du point de vue des Blancs (score positif favorable aux Blancs, négatif favorable aux Noirs). Elle combine plusieurs composantes, chacune calculée par une sous-fonction dédiée dans `custom_ai.py` :

Matériel	Pion=1, Cavalier/Fou=3, Tour=5, Dame=9, Roi=0 (poids classiques)
Contrôle du centre	Bonus par pion occupant une case centrale (d4/d5/e4/e5) ou le centre élargi
Cavalier au bord	Pénalité si un cavalier est posté sur la première/dernière colonne ou rangée
Tours liées	Bonus si les deux tours d'une couleur se voient (même rangée/colonne, rien entre elles)
Développement	Bonus par cavalier/fou ayant quitté sa case de départ
Sécurité du roi	Bonus si le roi est roqué avec un bouclier de pions devant lui ; malus s'il reste au centre en milieu de partie

Les cas de mat, pat, nulle par matériel insuffisant, règle des 50 coups et répétition sont traités en amont : un mat renvoie un score extrême (± 100000) et les autres issues nulles renvoient 0, indépendamment des critères positionnels.

3.3 Élagage Alpha-Bêta

La fonction `minmax(board, depth, alpha, beta, joueur_max)` explore récursivement les coups légaux de la position courante. À chaque niveau, α (meilleure valeur garantie pour le joueur maximisant) et β (meilleure valeur garantie pour le joueur minimisant) sont mis à jour au fur et à mesure de l'exploration des coups frères. Dès que $\alpha \geq \beta$, la branche restante est coupée (break) : l'adversaire dispose déjà d'une alternative au moins aussi bonne ailleurs dans l'arbre, donc il est inutile de continuer à explorer cette branche pour affiner sa valeur exacte.

Complexité : un MinMax naïf explore $O(b^d)$ nœuds (b = facteur de branchement, d = profondeur). Avec un élagage Alpha-Bêta efficace, la complexité peut descendre jusqu'à $O(b^{(d/2)})$ dans le meilleur des cas, ce qui permet en pratique d'atteindre une profondeur de recherche plus grande dans le même temps de calcul.

4. Usage de l'IA générative (section dédiée)

Conformément au règlement du TP, cette section documente précisément l'usage d'un assistant IA

(Claude, Anthropic) au cours du développement. Tout le code concerné est encadré dans le code source par les commentaires « `# ##### CODE IA (Claude) #####` » / « `### CODE IA (Claude) ###` ».

4.1 Pourquoi l'IA a été utilisée

- Toute la coquille du jeu (interface graphiques, les règles, ...)
- Enrichir la fonction d'évaluation heuristique avec des critères positionnels usuels aux échecs (au-delà du simple matériel), difficiles à écrire correctement à la main du premier coup.
- Ajouter un mécanisme d'adaptation de la profondeur de recherche selon la phase de la partie.
- Instrumenter le code de recherche (minmax / `get_best_move_custom`) avec le `tree_logger` fourni, pour brancher le visualiseur d'arbre bonus.
- La documentation

4.2 Prompts utilisés

Prompt 1 — Enrichissement de la fonction d'évaluation :

Prompt : *Modifie mon evaluation de l'echequier pour qu'elle prenne en compte les points suivants : récompenser quand les pions prennent le centre, punir quand le cavalier est au bord de l'echequier, recompenser quand les tours sont liées, recompenser quand toutes les pieces sont développées, recompenser quand le roi est à l'abri (roqué avec des pions devant lui)*

Prompt 2 — Profondeur de recherche dynamique :

Prompt : *Tu peux augmenter la profondeur en fonction du nombre de pieces qu'il reste sur l'echequier stp*

Prompt 3 — Instrumentation du visualiseur d'arbre : les appels à `tree_logger.new_node` / `update_node` ont été ajoutés dans `get_best_move_custom` et `minmax` en suivant l'exemple d'intégration fourni dans le fichier `ai/tree_logger.py` (docstring du module, section « exemple d'utilisation »).

4.3 Ce qui a été modifié / compris / validé

- Les cinq sous-fonctions d'évaluation générées (`_center_control_score`, `_knight_rim_penalty`, `_connected_rooks_score`, `_development_score`, `_king_safety_score`) ont été relues et comprises une à une : chacune isole un seul critère positionnel et retourne un score signé (positif pour les Blancs, négatif pour les Noirs), ce qui permet de les activer/désactiver ou pondérer indépendamment.
- La fonction `get_dynamic_depth` a été vérifiée manuellement sur quelques positions (nombre de pièces décroissant → profondeur croissante par paliers : 3 en ouverture, jusqu'à 6 en finale avancée).
- L'algorithme MinMax / Alpha-Bêta a été écrit et validé par l'étudiant, en s'appuyant sur le cours ; l'IA n'a été utilisée que pour les briques additionnelles listées ci-dessus.
- Le fonctionnement de l'élagage a été vérifié empiriquement en comparant, via le visualiseur d'arbre, le nombre de nœuds explorés avec et sans la condition de coupure sur quelques positions de test.

5. Données et fichiers nécessaires

Le projet ne nécessite aucune donnée externe ni fichier de configuration particulier : la position de départ est générée par python-chess et aucune base de données d'ouvertures ni table de fin de partie n'est utilisée. Seules les dépendances listées dans requirements.txt doivent être installées (voir README.md, section Installation).

6. Autres informations utiles

6.1 Tests

Les tests (tests/test_custom_ai.py) vérifient que `get_best_move_custom` renvoie systématiquement un coup appartenant à `board.legal_moves`, en position de départ et après une courte séquence d'ouverture. Ils s'exécutent avec la commande `pytest tests/`.

6.2 Limites connues et pistes d'amélioration

- Pas de gestion de timeout par coup : la profondeur dynamique peut rendre certains coups de finale plus longs à calculer.
- Absence de table de transposition et de tri des coups (move ordering), qui permettraient de réduire encore davantage le nombre de nœuds explorés à profondeur égale.
- Promotion toujours automatique en Dame (pas de choix de pièce à l'écran).
- Amélioration de l'algorithme d'évaluation (ex: récompenser si la tour est sur une colonne ouverte / semi-ouverte, récompenser quand on a une batterie Dame/Fou ou Dame/tour, ...)

6.3 Suivi Git

Le dépôt Git du projet trace l'avancement du développement via des commits réguliers, conformément aux exigences minimales du TP.